Experiment No.: 05

Experiment Title

Implementation of Breadth First Search (BFS) to Solve the 8-Puzzle Problem.

Aim

To implement the Breadth First Search (BFS) algorithm for solving the 8-Puzzle Problem and to find the shortest path from the initial state to the goal state.

Objective

- To understand the concept of state space search.
- To study the Breadth First Search (BFS) algorithm.
- To solve the 8-Puzzle Problem using BFS.
- To explore the shortest path solution in a state space.
- To analyze the performance of BFS in problem-solving.

Theory

Artificial Intelligence (AI) aims to develop intelligent systems capable of solving complex problems by searching through a set of possible states. Search techniques are fundamental to AI because many real-world problems can be represented as a state space where each state represents a possible configuration of the problem.

The **8-Puzzle Problem** is one of the most popular examples used in Artificial Intelligence to demonstrate state space search techniques. The puzzle consists of a 3×3 board containing eight numbered tiles and one blank space. The blank space can be moved in four possible directions (up, down, left, right) by swapping it with an adjacent tile.

The objective of the puzzle is to transform an initial arrangement of tiles into a predefined goal arrangement by making a sequence of valid moves.

State Space Representation

A state in the 8-puzzle represents the arrangement of all tiles on the board. Each movement of the blank tile generates a new state. The collection of all possible states forms the state space.

Example:

Initial State:

```
1 2 3
4 0 6
7 5 8
```

Goal State:

```
1 2 3
4 5 6
7 8 0
```

Here, '0' represents the blank tile.

Breadth First Search (BFS)

Breadth First Search is an uninformed search algorithm that explores nodes level by level. It visits all nodes at the current depth before moving to the next depth level.

BFS uses a **Queue (FIFO – First In First Out)** data structure. The algorithm starts from the initial state and systematically explores all neighboring states until the goal state is reached.

Working of BFS

1. Insert the initial state into the queue.
2. Remove the front node from the queue.
3. Check whether it is the goal state.
4. If not, generate all possible successor states.
5. Insert unvisited successor states into the queue.
6. Repeat the process until the goal state is found.

Because BFS explores states level by level, it guarantees the shortest solution path when all moves have equal cost.

Characteristics of BFS

- Uninformed search technique.
- Uses FIFO queue.

- Complete algorithm.
- Optimal for equal-cost paths.
- Explores states level by level.
- Guarantees shortest path solution.

Advantages of BFS

- Simple and easy to implement.
- Guarantees finding a solution if one exists.
- Finds the shortest path in an unweighted graph.
- Suitable for small and medium-sized search problems.

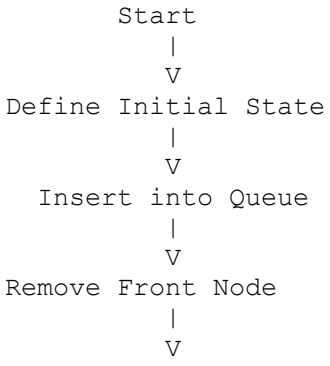
Limitations of BFS

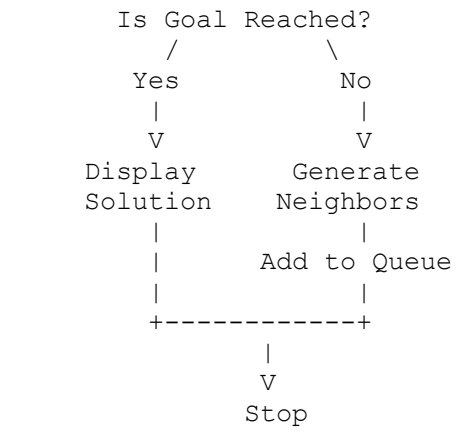
- Requires large memory space.
- May become slow for large state spaces.
- Stores all generated nodes in memory.
- Not suitable for very complex problems.

Algorithm

1. Start.
2. Define the initial state and goal state.
3. Create an empty queue.
4. Insert the initial state into the queue.
5. Remove the front state from the queue.
6. Check if it matches the goal state.
7. If yes, display the solution and stop.
8. Otherwise, generate all possible neighboring states.
9. Add unvisited neighboring states to the queue.
10. Repeat steps 5–9 until the goal state is found.
11. Stop.

Flowchart





Program

```
from collections import deque

start = [1, 2, 3,
         4, 0, 6,
         7, 5, 8]

goal = [1, 2, 3,
        4, 5, 6,
        7, 8, 0]

def neighbors(state):
    moves = {
        0:[1,3], 1:[0,2,4], 2:[1,5],
        3:[0,4,6], 4:[1,3,5,7], 5:[2,4,8],
        6:[3,7], 7:[4,6,8], 8:[5,7]
    }

    pos = state.index(0)
    result = []

    for m in moves[pos]:
        new = state[:]
        new[pos], new[m] = new[m], new[pos]
        result.append(new)

    return result

queue = deque([start])
visited = []

while queue:
    current = queue.popleft()

    print(current)

    if current == goal:
        print("\nGoal Reached")
        break
```

```
    visited.append(current)

    for n in neighbors(current):
        if n not in visited and n not in queue:
            queue.append(n)
```

Sample Output

```
[1, 2, 3, 4, 0, 6, 7, 5, 8]
[1, 0, 3, 4, 2, 6, 7, 5, 8]
[1, 2, 3, 0, 4, 6, 7, 5, 8]
[1, 2, 3, 4, 6, 0, 7, 5, 8]
[1, 2, 3, 4, 5, 6, 7, 0, 8]
[1, 2, 3, 4, 5, 6, 7, 8, 0]

Goal Reached
```

Observation

Step	State Explored
1	Initial State
2	Neighbor States Generated
3	Queue Updated
4	Goal State Found

Applications

1. Puzzle solving (8-Puzzle, 15-Puzzle).
2. Robot path planning.
3. Navigation systems.
4. Network routing algorithms.
5. Social network analysis.
6. Web crawling and indexing.
7. Artificial Intelligence problem solving.
8. Game state exploration.

Result

The Breadth First Search (BFS) algorithm was successfully implemented to solve the 8-Puzzle Problem. The algorithm explored states level by level and found the goal state while guaranteeing the shortest solution path.

Viva Questions

1. What is Breadth First Search?
2. What is a state space?
3. What is the 8-Puzzle Problem?
4. Why is BFS called an uninformed search?
5. Which data structure is used in BFS?
6. What is the difference between BFS and DFS?
7. Is BFS complete?
8. Is BFS optimal?
9. What is a goal state?
10. What is a successor state?
11. What is a search tree?
12. What are the limitations of BFS?
13. What is the time complexity of BFS?
14. What is the space complexity of BFS?
15. What are the applications of BFS?

Conclusion

The experiment successfully demonstrated the implementation of Breadth First Search for solving the 8-Puzzle Problem. BFS systematically explored all possible states level by level and successfully reached the goal state, proving its effectiveness in solving planning and state space search problems.

Or

Aim

To implement the Breadth First Search (BFS) algorithm for Robot Path Planning and find the shortest path from the start position to the goal position.

Objective

- To understand the concept of robot path planning.
- To study Breadth First Search (BFS) as a search technique.
- To find the shortest path between two locations.
- To apply BFS in a grid-based environment.
- To analyze the efficiency of BFS in path planning problems.

Theory

Robot Path Planning is one of the important applications of Artificial Intelligence. The objective of path planning is to determine a valid and optimal path for a robot from a starting position to a destination while avoiding obstacles.

In AI, path planning problems can be represented as a graph where:

- Each cell or location is considered a node.
- Possible movements between cells are considered edges.
- The robot searches for a path from the start node to the goal node.

Breadth First Search (BFS) is an uninformed search algorithm that explores nodes level by level. BFS uses a Queue (FIFO) data structure and guarantees the shortest path when all movements have equal cost.

In Robot Path Planning, BFS starts from the robot's initial position and explores all neighboring positions. It continues expanding nodes until the goal position is reached. Since BFS explores nodes level by level, the first path found is always the shortest path.

Characteristics of BFS

- Uninformed search algorithm.
- Uses FIFO Queue.
- Complete search method.
- Guarantees shortest path.
- Suitable for grid and graph traversal.

Advantages

- Simple to implement.
- Finds shortest path.
- Complete and optimal for equal-cost paths.
- Widely used in robotics and navigation.

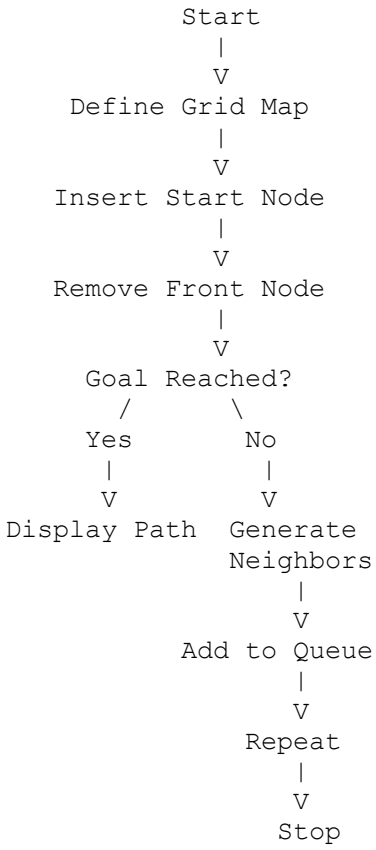
Limitations

- Requires large memory.
- Slow for large search spaces.
- Explores many unnecessary nodes.

Algorithm

1. Start.
2. Define the grid environment.
3. Specify start and goal positions.
4. Insert the start position into the queue.
5. Remove the front node from the queue.
6. If the node is the goal, stop.
7. Generate valid neighboring positions.
8. Add unvisited neighbors to the queue.
9. Mark visited positions.
10. Repeat until the goal is found.
11. Display the shortest path.
12. Stop.

Flowchart



Program

```
from collections import deque

grid = [
```

```
    [0, 0, 0, 0],
    [1, 1, 0, 1],
    [0, 0, 0, 0],
    [0, 1, 1, 0]
]

start = (0, 0)
goal = (3, 3)

queue = deque([(start, [start])])
visited = set()

while queue:
    (x, y), path = queue.popleft()

    if (x, y) == goal:
        print("Path Found:")
        print(path)
        break

    visited.add((x, y))

    for dx, dy in [(0,1),(1,0),(0,-1),(-1,0)]:
        nx, ny = x + dx, y + dy

        if (0 <= nx < 4 and 0 <= ny < 4 and
            grid[nx][ny] == 0 and
            (nx, ny) not in visited):

            queue.append(((nx, ny), path + [(nx, ny)]))
```

Sample Output

```
Path Found:
[(0,0), (0,1), (0,2), (1,2),
 (2,2), (2,3), (3,3)]
```

Observation Table

Step	Current Position	Action
1	(0,0)	Start
2	(0,1)	Move Right
3	(0,2)	Move Right
4	(1,2)	Move Down
5	(2,2)	Move Down

6	(2,3)	Move Right
7	(3,3)	Goal Reached

Applications

1. Autonomous robot navigation.
2. Mobile robot path planning.
3. Warehouse automation systems.
4. GPS and route navigation.
5. Game AI movement planning.
6. Maze solving problems.
7. Drone navigation systems.
8. Industrial robotics.

Result

The Breadth First Search (BFS) algorithm was successfully implemented for Robot Path Planning. The algorithm explored nodes level by level and found the shortest path from the start position to the goal position while avoiding obstacles.

Viva Questions

1. What is Robot Path Planning?
2. What is Breadth First Search?
3. Why is BFS suitable for path planning?
4. Which data structure is used in BFS?
5. What is a node in a graph?
6. What is a goal state?
7. What is a visited node?
8. What is the advantage of BFS?
9. Is BFS complete?
10. Is BFS optimal?
11. What is the role of a queue in BFS?
12. What are obstacles in robot navigation?
13. What is the time complexity of BFS?
14. What is the space complexity of BFS?
15. Name some applications of BFS.

Conclusion

The experiment successfully demonstrated the use of Breadth First Search for Robot Path Planning. BFS efficiently explored the environment, avoided obstacles, and determined the shortest path between the start and goal positions.